

APOSTILA COMPLETA

Expressões, Statements & Métodos em Java

Do conceito de método até sobrecarga e desafios práticos

Expressões • Métodos • Parâmetros • Retorno • Sobrecarga

7 seções • Métodos e parâmetros • Sobrecarga e boas práticas

Complete Java Masterclass

Sumário

01	Unidades de Código Java	Expressões, statements, blocos e whitespace
02	Introdução aos Métodos	O que são, benefícios, estrutura básica
03	Parâmetros vs. Argumentos	Definição, regras, chamadas válidas e inválidas
04	Tipo de Retorno e void	return, assinatura do método e o método main
05	if-then-else Completo	if simples, else, else-if encadeado
06	Sobrecarga de Métodos	Overloading, assinatura, regras de validade
07	Desafios e Boas Práticas	High Score, Segundos/Minutos, boas práticas

CAPÍTULO 01

Unidades de Código Java

Expressões, statements e blocos — a hierarquia do código

As Três Unidades Fundamentais

Escrever código Java é como escrever um documento estruturado — ele tem unidades hierárquicas. No nível mais baixo estão as expressões, que se combinam para formar statements, que se agrupam em blocos, que se organizam em métodos, que se encapsulam em classes.

```
// EXPRESSÃO – computa e retorna um valor:

2 + 3 // retorna 5

idade >= 18 // retorna true ou false

nome.length() // retorna int (tamanho da string)


// STATEMENT – unidade de trabalho completa (termina com ;):

int x = 5; // declaration statement

x = x + 1; // expression statement

System.out.println("Olá"); // method call statement


// BLOCO – grupo de statements entre {}:

if (x > 0) {

    System.out.println("Positivo");

    x = 0;

}
```

■ **Dica:** Whitespace (espaços, tabs, linhas em branco) é completamente ignorado pelo compilador Java. Use-o generosamente para tornar o código legível — siga o Google Java Style Guide.

CAPÍTULO 02

Introdução aos Métodos

O que são métodos, por que usar e estrutura básica

Por que usar Métodos?

Um método é uma forma de agrupar código executável para evitar repetição. Sem métodos, você seria obrigado a copiar e colar o mesmo código em vários lugares — qualquer correção futura precisaria ser feita em todos os lugares, aumentando o risco de bugs. Métodos tornam o código modular, reutilizável e fácil de manter.

Um método pode ser chamado muitas vezes com argumentos diferentes, produzindo resultados diferentes a cada chamada. Isso é o princípio DRY (Don't Repeat Yourself) — escreva uma vez, use muitas vezes.

```
// Método mais simples – sem parâmetros, sem retorno:

public static void calcularPontuacao() {

    System.out.println("Calculando...");

}

// Chamando o método:

calcularPontuacao();

// Método com parâmetros e retorno:

public static int calcularPontuacao(boolean ganhou, int nivel, int bonus) {

    if (ganhou) {

        return nivel * 1000 + bonus;

    }

}
```

```
return 0;
```

```
}
```

```
// Chamando com argumentos:
```

```
int total = calcularPontuacao(true, 3, 500); // total = 3500
```

CAPÍTULO 03

Parâmetros vs. Argumentos

A diferença, regras obrigatórias e chamadas corretas

Definições

Parâmetro é a variável definida na declaração do método — o que o método espera receber. Argumento é o valor real passado ao chamar o método. Essa distinção de terminologia é importante para a comunicação técnica clara.

```
// PARÂMETROS estão na DECLARAÇÃO:

public static void saudar(String nome, int idade) {

    System.out.println("Olá, " + nome + "! " + idade + " anos.");

}

// ARGUMENTOS estão na CHAMADA:

saudar("Ana", 25); // "Ana" e 25 são argumentos

// ■ Erros comuns ao chamar:

// saudar(25, "Ana"); // ordem errada!

// saudar("Ana"); // argumentos insuficientes!

// saudar("Ana", 25, 30); // argumentos demais!
```

■ ■ **Atenção:** Java NÃO suporta parâmetros com valor padrão como Python ou JavaScript. Você deve sempre passar todos os argumentos. Para ter valores padrão, use sobrecarga de métodos (overloading).

CAPÍTULO 04

Tipo de Retorno e void

Como declarar, quando usar return e a assinatura do método

Declarando o Tipo de Retorno

O tipo de retorno é declarado antes do nome do método. **void** significa que o método não retorna nenhum dado — apenas executa código. Para qualquer outro tipo (int, double, String, boolean, etc.), o método DEVE ter um **return** com um valor compatível em TODOS os caminhos de execução possíveis.

```
// void – nenhum retorno:

public static void imprimirMensagem(String msg) {

    System.out.println(msg);

    // return; // opcional em void – pode encerrar antecipado

}

// String – retorno em todos os caminhos:

public static String classificar(int nota) {

    if (nota >= 7) return "Aprovado";

    else return "Reprovado"; // TODOS os caminhos retornam

}

// Padrão recomendado – variável resultado:

public static int maximo(int a, int b) {

    int resultado = a; // valor padrão
```

```
if (b > a) resultado = b;  
  
return resultado; // único return no final  
  
}
```

■ **Dica:** Boa prática: declare um valor padrão no início do método e use um único return no final. O código fica mais previsível, mais fácil de debugar e testar.

CAPÍTULO 05

if-then-else Completo

Estrutura com múltiplas condições e else-if encadeado

As Três Formas do if

O Java suporta três formas de estrutura condicional: o if simples (executa ou não executa), o if-else (uma coisa ou outra) e o if-else-if encadeado (múltiplas alternativas). A última condição else (sem parâmetro) é opcional e captura todos os casos não cobertos pelas condições anteriores.

```
// 1. if simples:

if (nota >= 9) { System.out.println("Excelente"); }


// 2. if-else:

if (nota >= 7) { System.out.println("Aprovado"); }

else { System.out.println("Reprovado"); }


// 3. if-else if-else (múltiplas condições):

if (nota >= 9) { System.out.println("Excelente"); }

else if (nota >= 7) { System.out.println("Aprovado"); }

else if (nota >= 5) { System.out.println("Recuperação"); }

else { System.out.println("Reprovado"); }
```

CAPÍTULO 06

Sobrecarga de Métodos (Overloading)

Mesmo nome, parâmetros diferentes — polimorfismo em compile time

O que é Sobrecarga?

Sobrecarga (overloading) é a capacidade de ter múltiplos métodos com o mesmo nome em uma classe, desde que tenham parâmetros diferentes (em tipo, quantidade ou ordem). O compilador decide qual versão chamar analisando os argumentos passados — isso é chamado de polimorfismo de compilação.

```
// Método 1: recebe polegadas totais

public static double converterParaCentimetros(int polegadas) {

    return polegadas * 2.54;

}

// Método 2: recebe pés e polegadas

public static double converterParaCentimetros(int pes, int polegadas) {

    int totalPolegadas = (pes * 12) + polegadas;

    return converterParaCentimetros(totalPolegadas); // chama método 1!

}

// Java identifica qual usar pelos argumentos:

double a = converterParaCentimetros(68); // 172.72 cm

double b = converterParaCentimetros(5, 8); // 172.72 cm
```

■ ■ **Atenção:** O tipo de retorno e os nomes dos parâmetros NÃO fazem parte da assinatura. Sobrecarga que difere apenas no tipo de retorno ou nos nomes dos parâmetros causa erro de compilação.

CAPÍTULO 07

Desafios e Boas Práticas

High Score, conversão de segundos e princípios de design de métodos

Desafio: High Score

```
public static void displayHighScorePosition(String nome, int posicao) {  
  
    System.out.println(nome + " entrou na posição " + posicao + "!");  
  
}  
  
public static int calculateHighScorePosition(int score) {  
  
    if (score >= 1000) return 1;  
  
    else if (score >= 500) return 2;  
  
    else if (score >= 100) return 3;  
  
    return 4;  
  
}
```

Desafio: Converter Segundos

```
public static String getDurationString(int segundos) {  
  
    if (segundos < 0) return "Inválido!";  
  
    return getDurationString(segundos / 60, segundos % 60);  
  
}  
  
public static String getDurationString(int minutos, int segundos) {  
  
    if (minutos < 0 || segundos < 0 || segundos > 59) return "Inválido!";  
  
}
```

```
return (minutos/60) + "h " + (minutos%60) + "m " + segundos + "s";

}

// Testes:

System.out.println(getDurationString(3945)); // 1h 5m 45s
```

Boas Práticas

- Um método deve fazer UMA coisa bem feita — princípio da responsabilidade única.
- Mantenha métodos curtos — idealmente menos de 20-30 linhas.
- Use nomes descritivos: `calcularMedia()`, não `cm()` ou `x()`.
- Valide entradas em métodos públicos — nunca confie nos argumentos recebidos.
- Prefira um único `return` no final para facilitar debug.

Parabéns por concluir a apostila!

Métodos são a base de todo código Java bem organizado. Dominar parâmetros, retornos e sobrecarga é fundamental para construir programas modulares e reutilizáveis.

Complete Java Masterclass